

2026-05-05-p1-f06-mcp-full-lifecycle-design

Phase 1 / Feature 6 — MCP Full Lifecycle (4 Transports + OAuth)

Date: 2026-05-05 **Status:** Approved (brainstorming) **Programme:** CLI-Agent Fusion — Phase 1
port from claude-code

1. Goal

Add client-side Model Context Protocol (MCP) capability to HelixCode so it can connect to external MCP servers across four transports (stdio, HTTP, SSE, WebSocket) with full OAuth 2.0 support (RFC 8414 Authorization Server discovery, PKCE, token refresh). External MCP servers' tools become first-class agent tools alongside built-in tools.

The existing server-side `internal/mcp/` package (HelixCode-as-MCP-server) is untouched.

2. Architecture

Extend `internal/mcp/` with client-side files alongside `server.go`. Reuse existing `MCPMessage/MCPError/Tool` JSON-RPC primitives (already exported from `server.go`). Client and server share the same package; role-prefixed file names keep them separable in review.

A single `Transport` interface is the only seam transports plug in through. `Client` per server holds transport, state machine, and pending-RPC bookkeeping. `Manager` aggregates clients across configured servers and exposes them to `internal/tools/registry.go` as agent-callable tools.

Configuration is YAML-first (project + user, project overrides). CLI mutation commands (`mcp add/remove`) round-trip the YAML so the file remains the single source of truth.

Boundary discipline: - Client code never imports anything from server-side beyond `MCPMessage/MCPError/Tool`. - No back-references from `server.go` into client code. - Adding a 5th transport later is one file under the same `Transport` interface.

3. Components

3.1 New files in `internal/mcp/`

File	Responsibility
<code>transport.go</code>	<code>Transport</code> interface, <code>TransportType</code> enum, <code>BackoffSchedule</code> helper
<code>transport_stdio.go</code>	Subprocess via <code>os/exec</code> ; stdin/stdout newline-delimited JSON-RPC

File	Responsibility
transport_stdio_unix.go	Setpgid + SIGKILL(-pid) for process-group cleanup (//go:build unix)
transport_stdio_windows.go	Job-object cleanup via <code>golang.org/x/sys/windows</code> (//go:build windows)
transport_http.go	JSON-RPC over POST request/response; OAuth bearer header
transport_sse.go	POST out, SSE in; auto-reconnect with <code>BackoffSchedule</code>
transport_ws.go	WebSocket via <code>gorilla/websocket</code> (already in deps)
oauth.go	RFC 8414 discovery, PKCE auth-code flow, token cache, refresh
lifecycle.go	Client struct, state machine, handshake, RPC dispatch
registry.go	Manager aggregating clients; tool merging; lifecycle
config.go	YAML loader/saver for <code>mcp.yml</code> (project + user)

3.2 New files outside `internal/mcp/`

File	Responsibility
<code>cmd/cli/mcp_cmd.go</code>	Cobra subcommands: add, list, remove, test, auth, logs
<code>internal/commands/mcp_command.go</code>	<code>/mcp</code> slash command for interactive CLI

3.3 Modified files

File	Change
<code>internal/tools/registry.go</code>	On init/refresh, query <code>mcp.Manager.Tools()</code> and register each as a <code>Tool</code> with handler routing to <code>Manager.CallTool</code> ; tool names prefixed <code><server>:<tool></code> to avoid collisions
<code>cmd/cli/main.go</code>	Wire <code>mcp.Manager.Start(ctx)</code> into startup; register <code>mcp_cmd</code> cobra command
<code>internal/commands/builtin/register.go</code>	Register <code>/mcp</code> slash command (mirrors / permissions, /worktree, /hooks)

3.4 Transport interface

```
type Transport interface {
    Open(ctx context.Context) error
```

```

    Send(ctx context.Context, msg *MCPMessage) error
    Recv(ctx context.Context) (*MCPMessage, error) // blocks;
        io.EOF on clean close
    Close() error
    Type() TransportType
}

```

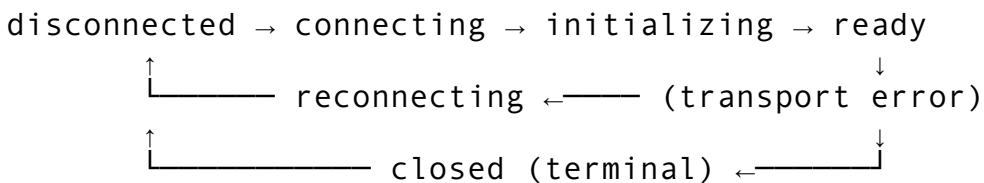
```

type TransportType string
const (
    TransportStdio TransportType = "stdio"
    TransportHTTP   TransportType = "http"
    TransportSSE    TransportType = "sse"
    TransportWS     TransportType = "ws"
)

```

BackoffSchedule: exponential 1s → 2s → 4s → 8s → 16s → 30s cap, ±20% jitter. Reset on successful handshake.

3.5 Client state machine



State stored as `atomic.Int32` for cheap /mcp status reads.

3.6 Manager API

```

type Manager struct {
    clients map[string]*Client
    config  *Config
    mu      sync.RWMutex
}

func (m *Manager) Start(ctx context.Context) error
func (m *Manager) Tools() []ExternalTool
func (m *Manager) CallTool(ctx context.Context, server, tool
    string, args map[string]any) (*CallResult, error)
func (m *Manager) Reload(ctx context.Context) error
func (m *Manager) Status() []ClientStatus
func (m *Manager) Test(ctx context.Context, name string) error
func (m *Manager) Logs(name string) ([]byte, []Event, error)
func (m *Manager) Close() error

```

3.7 YAML schema

```

servers:
  - name: brave-search

```

```

transport: stdio
command: ["npx", "-y", "@modelcontextprotocol/server-brave-
  search"]
env:
  BRAVE_API_KEY: ${BRAVE_API_KEY}
alwaysLoad: true

- name: cloudflare
  transport: sse
  url: https://mcp.cloudflare.com/sse
  oauth:
    enabled: true
  alwaysLoad: false

```

alwaysLoad: true connects at startup. alwaysLoad: false lazy-connects on first tool call.

3.8 CLI surface

```

helixcode mcp add <name> --transport=stdio --command="npx ..."
helixcode mcp add <name> --transport=sse --url=https://... [--oauth]
helixcode mcp list                                     # name, transport, status, tool c
helixcode mcp remove <name>
helixcode mcp test <name>                             # connect → initialize → tools/li
helixcode mcp auth <name>                             # OAuth PKCE flow, persist token
helixcode mcp logs <name>                             # last 64KB stderr (stdio) + life

```

/mcp slash command: list current clients with state and tool count; supports /mcp test <name>, /mcp logs <name>, /mcp reload.

4. Data flow

4.1 Startup

```

helixcode launch
├─ mcp.Manager.Start(ctx)
│   ├─ load YAML (user, then project overrides)
│   └─ for each alwaysLoad=true server (parallel):
│       ├─ build Transport from config
│       └─ Client.Connect()
│           ├─ transport.Open
│           ├─ send initialize request
│           ├─ recv response, store capabilities
│           ├─ send notifications/initialized
│           ├─ send tools/list, store tools
│           └─ state = ready
└─ tools.Registry.Refresh() picks up Manager.Tools()

```

Lazy-load servers connect on first CallTool.

4.2 Tool invocation

```
agent picks "brave-search:search"
└─ tools.Registry → mcp.Manager.CallTool("brave-search", "search", args)
    └─ if not ready → Connect (lazy) or wait for reconnect
        └─ assign rpc id, register pending channel
            └─ transport.Send(tools/call)
                └─ wait on pending channel (ctx-cancellable)
                    └─ return result.content / result.isError
```

ErrReconnect returned if mid-reconnect; tool layer can retry.

4.3 Reconnect (SSE / WS)

```
Recv loop sees transport error
└─ state = reconnecting
    └─ fail all pending RPCs with ErrReconnect
        └─ backoff loop:
            └─ wait BackoffSchedule.Next()
                └─ transport.Close, recreate
                    └─ transport.Open
                        └─ handshake (initialize → initialized → tools/list)
                            └─ state = ready, resume
```

4.4 OAuth flow (mcp auth <name>)

1. Read server config; require transport=http|sse, oauth.enabled
2. oauth.Discover(ctx, baseUrl) → AS metadata
3. Generate PKCE verifier+challenge, state, port
4. Spin one-shot http.Server on 127.0.0.1:<port>/callback
5. Open browser to authorization_endpoint?...
6. User authenticates; AS redirects to callback with code
7. Exchange code at token_endpoint (with verifier)
8. Persist tokens to ~/.config/helixcode/mcp/tokens/<name>.json (mode 0600)
9. Tokens flow into transport via oauth2.TokenSource on next connect

On 401 mid-call: oauth2.TokenSource auto-refreshes; persist new token.

4.5 Reload

Manager.Reload(ctx): - Re-read YAML, diff against current config. - Removed:
Client.Close(), drop from map. - Added: build Client; Connect if alwaysLoad. - Changed:
Close old, build new, Connect if alwaysLoad. - tools.Registry.Refresh().

4.6 Shutdown

Manager.Close() calls Client.Close() for every client in parallel (10s timeout); each cancels backoff loop, closes transport, drops pending channels.

5. Error handling

5.1 Error taxonomy

```
var (  
    ErrServerNotFound = errors.New("mcp: server not found")  
    ErrNotReady       = errors.New("mcp: client not ready")  
    ErrReconnect      = errors.New("mcp: transport  
        reconnecting")  
    ErrInitFailed     = errors.New("mcp: initialize handshake  
        failed")  
    ErrTransportClosed = errors.New("mcp: transport closed")  
    ErrOAuthRequired  = errors.New("mcp: oauth token missing or  
        invalid; run 'helixcode mcp auth'")  
    ErrToolNotFound   = errors.New("mcp: tool not found on  
        server")  
    ErrProtocol       = errors.New("mcp: protocol violation")  
    ErrTooManyPending = errors.New("mcp: too many pending  
        requests")  
)
```

JSON-RPC error responses (MCPError) are wrapped with `fmt.Errorf("mcp call %s: %w", method, err)` preserving code/message/data.

5.2 Transport-specific failure modes

- **stdio**: subprocess exits non-zero → state=disconnected; last 64KB stderr surfaced via mcp logs. SIGKILL on process group (Unix) or job object (Windows). No reconnect.
- **http**: non-2xx → wrap as MCPError; 401 → ErrOAuthRequired if `oauth.enabled`, else ErrProtocol. No reconnect.
- **sse**: stream EOF/error → reconnect loop. POST 4xx → ErrProtocol; 401 → token refresh + retry once.
- **ws**: ping/pong timeout (30s) or close frame → reconnect. 401 handling matches SSE.

5.3 OAuth edge cases

- Discovery missing → fall back to config-provided endpoints; otherwise ErrOAuthRequired.
- Token expired mid-call → auto-refresh; refresh failure → ErrOAuthRequired, mark disconnected.
- Headless mcp auth → print authorization URL; 5-min callback timeout; documented limitation.
- Token cache mode != 0600 → refuse to read; refuse to start client.
- Tokens never logged.

5.4 Concurrency invariants

- One Recv loop per client (single goroutine per transport).
- pending map: `sync.Mutex`-guarded, channels buffered size 1.
- `Manager.clients`: `sync.RWMutex`; `Tools()/CallTool` read lock; `Reload` write lock.

- `Client.state: atomic.Int32`.
- `BackoffSchedule.Next()`: monotonic; reset on successful handshake.

5.5 Anti-bluff (CONST-035, Article XI §11.9)

- `mcp test <name>` PASSEs only on real connect → initialize → tools/list → close with non-empty server capabilities.
- Challenge spawns a real reference MCP server (`@modelcontextprotocol/server-everything` from npm), connects via stdio, calls a tool, verifies response. Pasted runtime evidence required.
- No transport returns PASS on metadata absence. SSE reconnect tests use a real test server that drops mid-stream.
- Anti-bluff smoke `grep -rn "simulated\|for now\|TODO implement\|placeholder" internal/mcp` must return empty.

5.6 Resource limits

- pending map cap: 1024 in-flight per client → `ErrTooManyPending`.
- Stdio stderr ring: 64KB, drops oldest.
- HTTP/SSE response body cap: 32MB.
- WebSocket message cap: 16MB.

5.7 Logging

- State transitions logged via existing `internal/logging`.
- JSON-RPC payloads at DEBUG only, with auth headers redacted.
- Lifecycle events emit on `onEvent` callback for `/mcp` and `mcp logs`.

6. Testing

6.1 Unit tests (mocks allowed)

- `transport_test.go` — backoff sequence + jitter; type validation; JSON round-trip for client methods.
- `transport_stdio_test.go` — fake subprocess via Go test helper; framing; stderr capture; SIGKILL on Close; exit-code surfacing.
- `transport_http_test.go` — `httptest.Server`; request shape; auth header; 4xx → `ErrProtocol`; 401 → `ErrOAuthRequired`.
- `transport_sse_test.go` — SSE parser handles multi-line data:, comments; reconnect after server closes mid-stream.
- `transport_ws_test.go` — read/write pumps; ping timeout → reconnect; close frame.
- `oauth_test.go` — discovery; PKCE verifier ≥43 chars + SHA256-base64url challenge; auth-code exchange; mode-0600 persistence; refresh on 401; refusal on bad mode.
- `lifecycle_test.go` — fake Transport; state machine; pending cap; `ErrReconnect` during reconnect.
- `registry_test.go` — fake clients; parallel Start; tool merging with name prefix; Reload diff; shutdown timeout.
- `config_test.go` — YAML parse; project-overrides-user; env-var expansion; validation errors.

6.2 Integration tests (-tags=integration, real subprocess + real HTTP)

- tests/integration/mcp_stdio_test.go — spawn @modelcontextprotocol/server-everything via npx; full Connect → tools/list → tools/call → Close; assert non-empty tools, real output.
- tests/integration/mcp_http_test.go — small Go HTTP MCP server in-test; full handshake + tool call.
- tests/integration/mcp_sse_test.go — Go MCP server with SSE; reconnect by tearing server mid-stream.
- tests/integration/mcp_ws_test.go — WS variant.
- tests/integration/mcp_oauth_test.go — local OAuth AS implementing RFC 8414 + PKCE; full flow + refresh.

All -tags=integration-gated.make test-integration-full runs them. No bare t.Skip().

6.3 Challenge

challenges/p1-f06-mcp-full-lifecycle/ (mirrors F02–F05):

1. Build bin/helixcode.
2. Write .helixcode/mcp.yml with one stdio server (@modelcontextprotocol/server-everything).
3. helixcode mcp test everything → exit 0, “ready” + tool count > 0.
4. helixcode mcp list → table includes the server.
5. Trigger one tool call via agent loop → real response (no “simulated”, no “for now”).
6. Anti-bluff smoke clean.
7. Cross-compile to Windows (G005=windows go build ./...) — proves cross-platform stdio compiles.

Pasted runtime evidence committed to docs/improvements/
06_phase_1_evidence.md per Article XI §11.9.

6.4 CLI tests

- cmd/cli/mcp_cmd_test.go — cobra parsing: add writes YAML, remove mutates, list reads, test invokes Manager.Test, auth runs flow with mocked browser opener, logs reads ring buffer.
- internal/commands/mcp_command_test.go — /mcp formatting; /mcp reload invokes Manager.Reload.

7. Cross-platform

shell_runner_unix.go / shell_runner_windows.go pattern from F05 applied to stdio process-group control:

- transport_stdio_unix.go — Setpgid + syscall.Kill(-pid, SIGKILL) (//go:build unix).
- transport_stdio_windows.go — job objects via golang.org/x/sys/windows (//go:build windows).

Each has a build tag and minimal platform-specific surface. Compile-tested on both platforms.

8. Out of scope (deferred)

- **Sampling** (`sampling/createMessage`) — protocol exists; defer to F06.5 if needed.
- **Roots** (`roots/list`) — publish working dir as the only root in initialize capabilities; full negotiation deferred.
- **Server-initiated tool calls** — handlers wired but no agent surface yet.
- **HelixAgent submodule auto-discovery** — registry of 40+ MCP submodules under `helix_agent/MCP/submodules` not auto-loaded; users opt in via YAML.

9. Constitutional compliance

- **CONST-033** (no power management): stdio transport never invokes shutdown/reboot; `Close` only kills the process group it started.
- **CONST-035 / Article XI §11.9** (anti-bluff): every PASS carries runtime evidence; challenge spawns a real MCP server and verifies real tool output.
- **CONST-036** (LLMsVerifier single source of truth): MCP server lists are user-configured YAML and CLI-managed; this feature does not introduce a separate model registry. Tool capabilities flow from `tools/list` JSON-RPC responses, not hardcoded.
- **CONST-039** (all-providers integration): MCP is orthogonal to LLM provider integration; this feature does not affect provider coverage.
- **CONST-042** (no-secret-leak): OAuth tokens persisted at mode 0600 under `~/.config/helixcode/mcp/tokens/`; `.gitignore` updated; tokens never logged.
- **CONST-043** (no-force-push): branch is `main`; commits pushed non-force to all four remotes per programme convention.

10. Open questions resolved during brainstorming

Question	Answer
Q1: extend existing vs new sub-package vs scope down	(A) Extend <code>internal/mcp/</code> with client-side files
Q2: which transports + OAuth	(A) All 4 transports + OAuth
Q3: config source	(C) YAML + CLI (YAML is source of truth, CLI round-trips)
Q4: CLI surface	(A) Full parity: <code>add</code> , <code>list</code> , <code>remove</code> , <code>test</code> , <code>auth</code> , <code>logs</code> , <code>/mcp slash</code>
Q5: JSON-RPC + transport implementation	(A) Hand-roll on <code>stdlib</code> + existing primitives, no new SDK